

OpenAI Agents API 科普拆解篇

—— sitin 《Agents API来了，Codex背后的这套能力终于开放了》一文精读 + 主张实测验证（增补篇）

拾遗任务 6628a24f · 2026-10-03 · 姊妹篇《OpenAIAgentsAPI-小白指南》（官方文档视角 12 问）已先行入库，本篇只补科普文独有增量

？ 工科毕业生的 10 个新疑问（疑问即目录）

1. "Agent $\approx$ Model + Harness"这个拆法靠谱吗？
2. harness 要回答的五问，官方四概念怎么接？
3. "长任务不是让 AI 一直想"是什么意思？
4. "别靠记忆硬撑"——产物链是什么？
5. Session 为什么能让任务不绑单次请求？
6. "工具不是越多越好"怎么落地？
7. 子 agent 什么时候并行、什么时候必须串行？
8. "验收标准写进任务"为什么比堆提示词有用？
9. Codex 和 Agents API 到底什么关系？
10. 这篇科普文和官方文档，先读哪个？

Q1 "Agent $\approx$ Model + Harness"，这个拆法靠谱吗？

靠谱，而且它就是本仓库已学七职责的入门版。Model（大模型）负责思考、推理、决策，但它不知道文件在哪、不会自己开终端；Harness（运行框架）负责组织上下文、工具调用、状态和子 agent，跑那个“模型判断 $\rightarrow$ 调工具 $\rightarrow$ 读结果 $\rightarrow$ 继续判断”的循环。



图1 Agent = 会思考的大脑 $\times$ 会办事的挽具

文章这套两分法，对应已学的 [[AgentHarness智能体挽具]] 七职责（契约 $\rightarrow$ 地图 $\rightarrow$ 最小权限工具 $\rightarrow$ 验证证据 $\rightarrow$ 上下文工程 $\rightarrow$ 失败廉价 $\rightarrow$ 停止条件）——科普版把七职责压成了“一句话版”，适合先记这句再展开。

实验 ex6 步骤1 实证：FakeModel 类里一个项目状态字段都没有——模型只输出决策 JSON，状态全在 Harness 里。“分工”是可以写进代码的架构事实，不是比喻。

Q2

harness 要回答的五问，官方四概念怎么接？

文章列了 harness 必须回答的五个问题：谁来保存任务状态？谁把文件、终端和浏览器交给它？工具调用失败以后从哪里继续？任务太大怎么拆子 agent？怎么留下过程让人检查？官方 Agents API 用四个概念——接住：

文章五问	官方概念	一句话接法
谁保存任务状态	Session (sess_id 持久对象)	任务在册，跨轮次不丢
谁交文件/终端/浏览器	Environment (沙箱三态)	工位可选：托管沙箱/你自己的机器
失败从哪继续	Events & Items + 托管压缩恢复	事件流水都在，retrieve 即续传
怎么拆子 agent	subagent (创建/打断/关闭也是 item)	harness 自己招临时工
怎么留过程	items / traces / artifacts	全过程可回放可复查

(四概念详见姊妹篇 Q3；本表证明科普文五问与官方架构**完全咬合，无虚构**)

Q3

"长任务不是让 AI 一直想"，是什么意思？

很多人把"长任务"理解成**让模型一次性思考几个小时**。文章给的正确图景更像做项目：**理解目标 → 拆步骤 → 做一半发现错就回去改 → 缺料再补 → 最后检查**。模型**不是在后台凭空思考，而是每次工具调用返回后，基于当前文件、终端输出和任务状态决定下一步**。

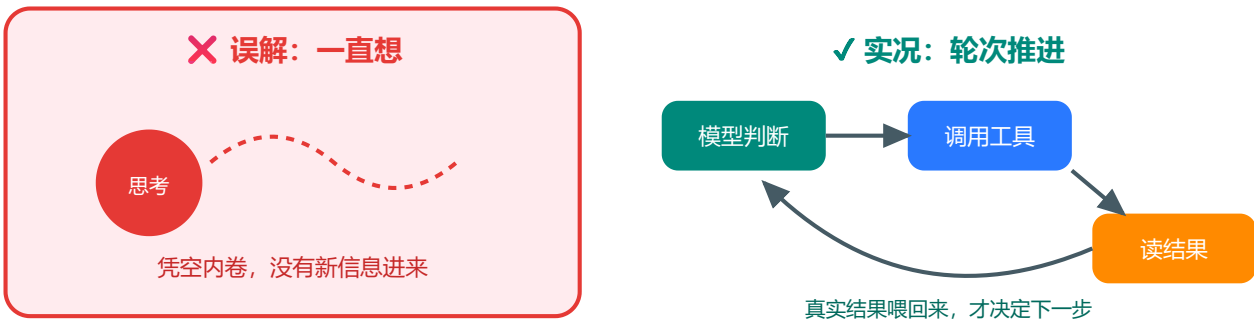


图2 "长任务"的正确打开方式：轮次循环，不是延长单次思考

以文章的老项目依赖升级为例，一条完整的轮次链是：**读仓库结构 → 盘点依赖 → 分批改配置 → 跑构建读报错 → 按错修复 → 查运行时行为 → 整理 diff 和遗留清单**——上一轮的产物（锁文件、报错日志）就是下一轮的输入。

Q4 "别靠记忆硬撑"——产物链是什么？

"不要让 AI 靠记忆硬撑，要让工作过程留下可以继续使用的东西。" ——全文金句

这条链路里每一步都会产生新东西：改过的配置、锁文件、修复的源码、测试日志、检查记录——**都成为后续行动可以读取的依据**。最后交付的也不是一句"升级完成"，而是一组**能让人复查的变更和结果**。我们把这个模式命名为**产物留痕与状态外置**：任务状态放在环境（文件/日志/产物）里，不放在模型上下文里。



图3 "记忆硬撑" vs "产物链" (ex6 步骤2: turn1_deps.txt / events.jsonl 11 条事件落盘, turn2 只读文件不靠回忆)

亲缘概念：[[Checkpoint存档与持久执行]]（框架替你快照对话）管"续跑"，**产物链**管"续跑时喂什么料"；[[JSONL事件日志与折叠模型]] 是它的日志实现。三者互链不重复。

Q5 Session 为什么能让任务不绑单次请求？

一次性请求 = 一问一答就散伙；Session = **一项任务自己的工作台**——消息、状态、工具调用、结果都围绕这个 session 积累。于是可以**中途插话改要求**（"先完成接口部分，页面暂时只给方案"），agent 收到后在**同一项任务上继续**。官方文档管这叫 **continue / steer**，与文章口径一致。

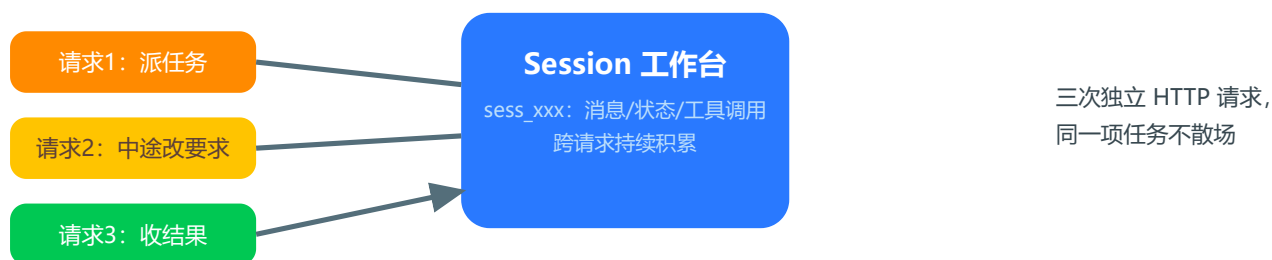


图4 任务不绑单次请求 (ex6 步骤3: 进程"重启"后从 session.json 恢复 turn=1 并注入追加要求)

与已学对照：[[Checkpoint存档与持久执行]] 是"存档+读档"，Session 是"在册员工不下班"——前者救崩溃，后者管协作。

Q6

"工具不是越多越好", 怎么落地?

按任务开放能力: 只需要读取数据, 就不应该顺手开放删除和发布; 任务只是整理文章, 就没必要给生产部署权限。涉及 **发布、发送、删除和付费操作**, **再单独由人确认**。"Agent 的能力边界越清楚, 长任务越容易放心地交出去。"

✗ 一串钥匙全给

读+写+删+发+付费 一口气授权
一次误操作 = 事故

✓ 按任务发钥匙

搜索任务→只给搜索; 改文件→限指定目录
删除/发布/付费→单独人工确认

图5 最小授权 (这是 [[AgentHarness智能体挽具]] 七职责里"最小权限工具"的科普版; [[提示注入]] 防线同款)

Q7

子 agent 什么时候并行、什么时候必须串行?

文章给出一条好用的判据: **彼此独立的部分才拆子 agent 并行** (查迁移文档 / 盘点调用点 / 查测试覆盖——输出可分别汇总); **共享文件的修改必须串行**, 由主 agent 统一安排顺序并验收, 否则"互相覆盖的修改或无法判断的冲突"。



图6 独立并行、共享串行 (ex6 步骤4: 两个只读调查并行完成序 b→a、2 份报告互不覆盖; shared_plan.md 串行追加 3 条)
亲缘概念: [[GitWorktree并行隔离]] (真要并行改文件就各开楼层)、[[私有子图输出隔离]] (各跑私有历史只回传报告键)、[[多Agent协作乱序竞态]] (完成序≠依赖序, ex6 实测 b 先于 a 完成就是活例)。

Q8

"验收标准写进任务"，为什么比堆提示词有用？

Agent 说"已经完成"时任务并没有结束。构建通过**只说明编译链路走通**，还要：跑测试和关键路径、查运行时日志、审核 diff 和锁文件、列出遗留项。文章的狠话：**"这一段往往比堆很多提示词更有用——它把任务的终点，从'AI 写完了'推进到'结果经过检查并且可以被人接手'。"**

这句话被 ex6 的 A/B 对照实验直接砸实：

步骤5（没写验收标准进配置）：模型自报 `report_done`

Gate 实测 → build PASS / runtime FAIL(RuntimeError: render_template was removed in miniflask 2.0) / leftovers FAIL → 拒收

步骤6（验收标准写进任务配置后返工）：4/4 全 PASS → 才允许报完成

✅ **预期结果（已实现）**：确定性 Gate 抓住了"构建通过但运行时已炸"的经典谎报——构建永远发现不了"render_template 在 2.0 被移除"这种运行时地雷。亲缘概念：[[证据优先质检ProofOverClaims]]（编译过不算数）、[[评分契约与护栏]]、[[规格收敛循环Converge]]（对账到 Converged 才算完）。

Q9

Codex 和 Agents API 到底什么关系？

文章的口径：**Codex 是已经做好的 Agent 工作环境**（打开项目、交任务、看过程和结果）；**Agents API 把支撑 Codex 的同一套 harness 和基础设施开放给开发者**——你要自己决定放哪个产品、给哪些工具、怎么展示状态、哪些操作等人确认。关键澄清：**"不意味着把 Codex App 嵌入自己的产品"**。选择很简单：只想让 AI 干本地活→直接用 Codex；想把这能力卖给你的用户→才研究 Agents API。

Codex = 成品餐厅

店面/后厨/服务员全配好
进门点单就吃，最省事



Agents API = 开放后厨

同一套后厨流水线租给你
菜谱（工具）/摆盘（UI）/收银（确认流）自己定
不是把整间餐厅搬进你的店

图7 三层运行时选型的科普版（要代驾→Agents API / 要方向盘→SDK / 只要发动机→Responses，详见姊妹篇 Q4）



这篇科普文和官方文档，先读哪个？

	sitin 科普文	官方文档 developers.openai.com
强项	认知框架（Model+Harness 五问、轮次论、产物链、验收论），10 分钟建立正确直觉	端点/参数/计费/安全边界，唯一权威；支持 URL 加 .md 抓纯文本
盲区	无一个可运行示例；计费/安全/三层选型全没讲	不告诉你"为什么"——新手容易迷路在参数里
建议	先读科普文建框架（本篇）→ 再按姊妹篇 12 问对官方文档逐项落地 → 动手只跑零 key 实验（ex1~ex6）	

事实核对表（文章主张 vs 存档官方文档 [agents-api-overview.md](#)）：managed harness ✓ / session 持久可 steer ✓ / subagent 支持 ✓ / 托管沙箱跑代码改文件 ✓ / 压缩恢复 ✓ ——**全部吻合，无虚构内容**。唯一保留：文末 HiAPI.ai 生图广告为作者自家产品推广，与技术内容无关。



主实验：200 行纯标准库 mini-harness，六步实测文章六大主张 (ex6_mini_harness.py, 零 key 零费用)

步骤1 | 把"Agent=Model+Harness"写成代码：FakeModel 类零状态字段，Harness 持有 session/tools/artifacts。

- ✓ 分工是架构事实，可编译验证

步骤2 | 产物链：模拟依赖升级，每轮产物落盘 artifacts/ + events.jsonl (11 条事件)。

- ✓ turn1_deps.txt = ['miniflask==2.0','requests==2.31'], 下一轮只读文件

步骤3 | Session 中断续接：保存 session.json → 进程"重启" → 恢复 turn=1 并注入追加要求"先只保证接口行为"。

- ✓ 任务不绑单次请求 (对照官方 steer/continue)

步骤4 | 子agent 并行/串行：两个只读调查线程并行 (模拟耗时 0.30s/0.10s)。

- ✓ 完成序 b→a (完成序≠启动序)、2 份报告互不覆盖；shared_plan.md 由主 agent 串行写 3 条

步骤5 | 验收 Gate 拒收谎报：模型自报完成 (build PASS)，Gate 真跑运行时检查。

- ✓ runtime FAIL(RuntimeError: render_template was removed in miniflask 2.0) + leftovers FAIL → 拒收

步骤6 | 验收标准写进任务 (A/B 对照)：按验收清单返工后重跑。

- ✓ 4/4 PASS 才报完成；EXIT=0，六大主张全部验证成立。过程踩坑实录：首版用真 flask 因环境未装而 ModuleNotFoundError 顶掉伏笔 → 改本地 miniflask 桩模块 (零依赖纪律)

📌 一页记忆卡 (科普拆解篇)

- **Agent = Model + Harness**: 大脑只决策, 挽具管循环/状态/工具/环境——可代码化验证
- **长任务 = 轮次推进**: 不是一直想, 是"工具结果喂回来再决定下一步"
- **产物留痕与状态外置** (新概念入库): 别靠记忆硬撑, 每轮产物是下一轮可读的依据
- **Session = 工作台**: 任务不绑单次请求, 中途可插话 (steer), 崩溃可恢复
- **工具按任务开放**: 最小授权 + 危险操作单独人工确认
- **子agent 判据**: 独立部分并行 (只读调查), 共享文件修改串行 (主 agent 统一写)
- **验收写进任务**: 构建通过≠完成; 确定性 Gate 抓谎报, 终点是"可被人接手"
- **Codex=成品餐厅 / Agents API=开放后厨**: 同一套 harness, 不是把 App 嵌进产品

拾遗任务 6628a24f · sitin X 文章 (2100394871265820672) 精读增补篇 · 实验脚本 openai-agents-api/exercise/ex6_mini_harness.py · 姊妹篇《OpenAIAgentsAPI-小白指南》(edc8f196) · 知识库: vault/概念/托管Harness与会话即资源 等